In LoginForm.jsx

<LoginButton isLoading={isLoading} />

How it works:

→ isLoading is passed from LoginForm through props.
→ if isLoading is True → the display Please wait...
→ if isLoading is False → the button display login
→ The parent component controls when loading starts & stops

LoginForm Manage isLoading
↓
Passes it to LoginButton through Props
↓
Button update its text and disabled state

The LoginButton doesn't manage loading by itself. It receives the loading state from its parent component

#Social login

allow user to login into an application using an existing account from another platform, such as google, github instead of creating a separate username and password

→ Social login act as a parent component that group different social login button into one section.

Sing in

Email



continue

OR

continue with google

continue with Github

```
function SocialLogin() {
  return (
    <div>
      <h3> Or Sign in with </h3>
      < GoogleLogin />
      < Github Login />
      </div>
```

Component

- SocialLogin.JSX
- GoogleLogin.JSX
- Github Login.JSX

Login.JSX

Social Login Button

provides a common button structure for different social login providers

Instead of writing separate button markup for every provider, we can create one component and then we can customize it using props.

SocialLoginButton.JSX

```
function SocialLoginButton ({ provider, onClick }) {
  return (
    <button onClick={onClick}>
      Continue with {provider}
    </button> ); }
export default SocialLoginButton;
```

GoogleLogin.JSX:

```
Import SocialLoginButton from "./SocialLoginButton";
function GoogleLogin () {
  return (
    <SocialLoginButton
      Provider = "Google"
      onClick = {() => console.log ("Google clicked")} />
    ); }
export default GoogleLogin;
```

Parent Component [google Login.JSX Passes props]

↓

Social Login Button [Receives Provider and onClick]

↓

Render Button [Display 'Continue with google']

↓

User Click Button [React trigger the OnClick handler]

↓

Authentication action [A configured function start google authentication]

OAuth 2.0 and OpenID Connect [OIDC]

OAuth 2.0 → is an authorization framework that allows application to obtain limited access to resources without asking the user for their provider password

OpenID Connect [OIDC] → is an identity layer built on top of oauth 2.0 it helps an application verify who the user it

User click
↓
React starts the configured login flow
↓
Google authenticates the user
↓
Application recieves authentication response.
(for OIDC, this can include an ID token)
↓
Response is validated securly
(The backend authentication library verifys it)
↓
User is signed in

OIDC use OAuth 2.0 mechanism and adds standardized identity information

M T W T F S S

Page No.: YOUVA

Date:

Provider configuration

Means setting up your application with a login provider such as google. So it can use that provides authentication service

Callback URL

Is the address where the authentication provider redirects the user after login process

http://localhost:5173/auth/callback

OTP Verification

→ Manages process of verifying user's identity using a OTP.

User enters login detail

↓

System sends or request OTP

↓

OTP verification displays the OTP form

↓

User enter the OTP and submit it.

↓

Valid OTP
continue verification or login

Invalid OTP.
display error

OTPInput.Jsx

```
import {useState} from 'react';
function OTPInput() {
  const [otp, setOTP] = useState("");
  return (
    <div>
      <input value={OTP} onChange={(e) => setOTP
      (e.target.value)} MaxLength={6}
      placeholder="Enter OTP" />
      <p>OTP: {OTP}</p> ); }

export default OTPInput;
```

```
function Login() {
  return (
    <div>
      <h2> OTP verification </h2>
      <OTPInput />
    </div> ); }

export default Login
```

OTPFORM Component

→ it organizes the OTP input field and the form submission process.

→ Display OTP input
→ Handles form submission
→ Checks whether the required OTP has be entered
→ Passes the OTP to a verification function

function OTPform() {

const [OTP, setOTP] = useState("");

function handleSubmit(e) {

e.preventDefault();

if(OTP.length !== 6) {

alert("Enter a 6 digit OTP");

return; }

alert(OTP ready for verification); }

return

<form onSubmit={handleSubmit}>

<input value={OTP} onChange={(e) => setOTP(e.target.value)}

maxLength={6}

placeholder="Enter OTP" />

<button type="submit">Verify OTP</button>

</form>); }

export default OTPform

→ Using it

import OTPform from "./OTPform";

function login() {

return

<h2>Verify your OTP Login</h2>

<OTPform />

};

}

M T W T F S S

Page No.:

Date:

YOUVA

M T W T F S S
Page No.:
Date: YOUVA

A Backend or authentication route to verify the OTP

Create OTPForm.JSX once.

↓

Import OTPForm into a Page

↓

Render <OTP Form />

↓

User enter and submit OTP

User Enter OTP in React

↓

React send Post / verify OTP

↓

Flask checks the submitted OTP

↓ ↓

Correct OTP — Return Success

Wrong OTP — Return an error

* Flask connect react ~~backend~~ ~~with~~ frontend with Python backend.

Resend OTP

→ allows users to request a new OTP if they haven't received the original one or it expired

→ Calls a backend API to generate and send new OTP.
→ display message when OTP is resent.
→ Can prevent repeated requests using a cooldown timer.

```
function ResendOTP() {
  function handleResend () {
  alleart ("Requesting a new OTP...") };
  }
  return (
    <button onClick = {handleResend} >
     Resend OTP.
    </button>
    );
    }
  export default ResendOTP;
```

User Click Resend OTP

↓

React send a request to backend.

↓

Backend generates and send a new OTP

↓

User receives a conformation message

Backend should generate and verify OTP securely. Enforce expiry and request limit and avoid sending to many OTPs in a short period

OTP Countdown:

→ display the remaining time before user can request another

Didn't receive the OTP?

00:30

You can request OTP after 30 seconds.

Resend OTP

```
function OTPCountdown() {
  const [seconds, setSecond] = useState(30);

  useEffect(() => { if (Second === 0) return;

  Const timer = SetTimeout(() => {
    Set Seconds((previous) => previous - 1); }, 1000);

  return () => ClearTimeout(Timer) ); }, [Seconds]));
return(
    <P2> { second > 0
          ? `Resend OTP in ${Seconds} Seconds`
          : "You can resend OTP"}
    </P> ); }

Export default OTPCountdown
```

UseState → Stores remaining seconds
UseEffect → runs the countdown logic when value change.
Set Timeout → Schedules the next one-second update.

OTP Sent
↓
Countdown starts at 30 sec.
↓
React updates the displayed time
↓
Timer reach Zero Resend Option available.

* The countdown controls user interface only. Flask must independently enforce resend limit because user can refresh the page.

Two factor Authentication

→ security feature that requires a user to verify their identity using a second method in addition to their password.

Step 1: Password
User enters their password.

Step 2: Second verification
User verifies with an OTP or authenticator app.

→ Helps protect account if a password is stolen
→ Can use an authenticator app, a security key or an OTP.

```
function 2FactorAuthentication() {
  const [otp, setOtp] = useState("");

function handleVerify(event) {
  event.preventDefault();

  alert("Send OTP to backend for verification");
return (
  <form onSubmit={handleVerify}>
    <h2>Two-factor Authentication</h2>
    <label>Enter your OTP</label>
    <form onSubmit
<input value={otp}
  onChange={(event) => setOtp(event.target.value)}
  maxLength={6}
  required placeholder="Enter OTP" />
<button type="submit">Verify</button>
```

</form>); }

export default 2 factor authentication

M T W T F S S
Page No.:
Date:
YOUVA

User enter username & password.

↓

Backend verifies password

↓

User completes second verification

↓ ↓

Verified (Login succeeds) | Failed (Access denied)

Two Factor Form.

→ Provides a form for user to complete second step of authentication such as entering OTP.

→ sends verification details to backend for checking.
→ display success or error message based on response

User Enters OTP

↓

React verified the input format

↓

Forms send OTP to backend.

↓

Backend verifies OTP and returns the result

* A form can validate the ~~input~~ input format, but only the backend should determine whether the OTP is valid

Register

→ represent the registeration page where a new user can create an account.

→ display registration page.
→ provide access to registration page
→ Can connect the form to a backend API to save user details

```
import RegistrationForm from "./RegistrationForm";

function Register () {
  return (
    <div>
      <h1> Create Account </h1>
      <p> Register to get Started </p>
    < RegistrationForm />
    </div>
  ); }
  export default Register;
```

User Open Registration Page
↓
Register display the Page
↓
Registration Form collects user details
↓
Backend validate and Save the details

* Register displays the page heading and renders a reusable RegistrationForm component.